Lecture 5

Software Architectural Process

Software Architecture Process

Outlines

- Architectural Process outline
- Architectural Design Methods
- Validation
- Design Quality Attribute
- SA Design Principles/Guidelines

Software Architectural Process

- The Software Architecture Process is the iterative series of activities that define, design, document, and maintain the structural organization of a software system.
- It acts as a blueprint, guiding development and ensuring that the system meets its functional and quality requirements.
- It bridging the gap between initial requirements and the final implemented system
- The architecture model and design should be completed before coding to guide development decisions on standards, technology, and implementation details.

Process Outline

Fundamental steps involved in the Software Architecture Process:

1. **Determine Architectural Requirements:**
   - This initial phase focuses on gathering and documenting all requirements that will shape the software architecture.
   - It involves into the SRS to understand both functional (what the system does) and non-functional (how the system performs) requirements.
   - Architects work closely with stakeholders to understand business goals, technical constraints, and user expectations.
   - Examples of architectural requirements:
     - **Performance:** "The system must respond to user requests within 2 seconds" (This impacts choices about database design, caching, etc.)
     - **Scalability:** "The system must handle up to 10,000 concurrent users" (This influences choices about server architecture, load balancing)

Process Outline

Fundamental steps involved in the Software Architecture Process:

2. **Identify Architecture Requirements:**
   - While the SRS provides a broad set of requirements, this step focuses on explicitly pinpointing those that have a *significant* impact on the software's architecture.
   - The focus is on distinguishing between general requirements and those that will have major impact on architectural choices (e.g., scalability needs, security protocols, integration points).

   Example:
   - SRS: "The system should be easy for novice users to learn.“
   - Architectural Requirement: "The system will employ a layered architecture with a separate presentation layer to facilitate UI changes" (This architectural decision supports ease of learning by allowing UI modifications without affecting core logic).

Process Outline

Fundamental steps involved in the Software Architecture Process:

3. **Prioritize Architecture Requirements:**
   - In real-world projects, resources and time are often limited, making it impossible to address every architectural requirement with equal emphasis.
   - Prioritization helps the architecture team focus on the most critical requirements that will have the greatest impact on the system's success.
   - Requirements are evaluated based on business value, technical feasibility, and risk factors.
   - Application of prioritization techniques: **MoSCoW Method** (Must-have, Should-have, Could-have, Won’t-have).

   Example:

   - In an e-commerce system, "Security" (protecting financial transactions) and "Scalability" (handling peak shopping times) might be prioritized over "Maintainability" (ease of code changes) in the initial architecture design.

S/W Development Life Cycle

1. Defining SRS (Software Requirements Specification) Stage

Objective: Capture and document what the software must do (system functions).

- **Tasks:**
  - Elicit **functional requirements** (features, user interactions).
  - Define **non-functional requirements** (performance, security, scalability).
  - Identify constraints (budget, timeline, regulatory).
- **Output**： SRS Document: A formal, unambiguous blueprint agreed upon by stakeholders.

S/W Development Life Cycle

2. System Analysis and Preliminary Design

Objective: Translate requirements into a technical blueprint.

- **Tasks:** System analysis and architectural design
  - **Architectural Design**: Choose patterns (e.g., MVC, microservices).
  - **Modeling**: Create UML diagrams (class, sequence, deployment).
  - **Hardware-Software Mapping**: Define infrastructure (cloud, servers, databases).
- **Output**： Preliminary design documents(high-level architecture).

S/W Development Life Cycle

3. Detailed Module Design

Objective: Specify how each component will work.

- **Task:**
  - Design APIs, data structures, and algorithms.
  - Define interfaces between modules (contracts for interaction).
  - Optimize for performance, security, and modularity.
- **Output:** Detailed Design Documents: Module specs, database schemas, pseudocode.

S/W Development Life Cycle

4. Code/Implementation

Objective: Build the software per design.

- **Task:**
  - Implement components(module) while following architectural guidelines.
  - **Integration testing** to ensure components work together.
  - Verify **traceability** (ensure code aligns with SRS).
- **Output:** Working Software: Testable components + integration builds.

S/W Development Life Cycle

5. Delivery and Sustaining Engineering

Objective: Ensure reliability post-deployment.

- **Tasks:**
  - **Testing**: Unit, integration, system
  - **Deployment**: Rollout strategies
  - **Maintenance**: Bug fixes, patches, updates.
  - **Engineering Support**: Monitoring, user training.
- **Output: Delivered Product** + **Maintenance Plans**.

Software Architecture (SA) Life Cycle

The SA life cycle includes:

1. **Non-formal Modeling** (Initial high-level design.).
2. **Analysis & Documented Design** on SA (Formalize architecture).
3. **Refining & Reviewing** on SA design (Validate against requirements).
4. **Implementation** of SA (Build the system).
5. **Evolution & Expansion** of SA (adapting to new requirements).
6. **Design Measurement & Improvement** (optimizing performance).
7. **End of Life Cycle** (retirement or replacement).

Things you need to know before designing

- Domain knowledge： business model/process, technical standards, core technology, etc.
- SRS understanding： deep understanding in functional and non-functional requirements
- Abstraction of system： overall structure, subsystems, external ports, etc.
- Architectural Model: arch style, key components, system constraints, etc.

Architectural Design Method

Choosing the Architecture Framework:

- This critical decision point involves selecting an appropriate architectural style or framework that best fits the system's requirements.
- The choice influences how components will interact, how the system will scale, and what constraints will be imposed on development.
- The framework selection process evaluates multiple dimensions:
  - Compatibility with target Architectural Style (layered, microservices, etc.)
  - Assessment of framework suitability for quality attributes
  - Consideration of organizational standards and team expertise
  - Documentation of architectural decisions and rationale

Architectural Design Method...

Choosing the Architecture Framework:

Several architectural design methods that serve as frameworks:

1. Component-based Architectural Design (CBAD)

   - Focuses on building the architecture from reusable components/module .
   - Uses OO modeling techniques
   - Promotes modularity and reusability but Less emphasis on overall system performance

Architectural Design Method...

Choosing the Architecture Framework..

2. Use Case-driven Architectural Design (UCAD)

- Build the SA model based on scenarios/use cases
- Architecture is derived from user scenarios (use cases) that defines user's functional requirements
- Uses UML for modeling.
- Ensures traceability to requirements but not appropriate for Architectural refinement and improvement

Architectural Design Method...

Choosing the Architecture Framework..

3. Domain-specific Software Architecture (DSSA)

- Starts from domain analysis and targets core domain functions.
- Tailored the architecture to a particular application domain for increased reuse
- Domain can be modeled by various attributes: class/component, framework, semantics/regulations, etc.
- Provides concrete solutions for specific domains
- Highly optimized for a particular industry but Less flexible for general-purpose applications.

Architectural Design Method...

Choosing the Architecture Framework..

4. Pattern-driven Architectural Design (PDAD)

- An advanced version of CBAD by emphasis on using architectural and component patterns.
- Emphasizes design reuse and Uses design patterns (e.g., MVC).
- Improves software quality (performance and maintainability)
- Good for certain type s/w applications such as SDK, middleware, framework, etc.

Architectural Design Method...

Choosing the Architecture Framework..

5. Model-driven Architecture (MDA)

- Focuses on architecture-implementation association.
- Employs models and automation to streamline the development process.
- Still need a component-based design method and well defined component/module interface.
- Use the CASE tools to generate code skeleton or framework API.
- Greatly improve design and code quality (Reduces manual coding errors).

Architectural Design Method...

Allocate Components

- Once the framework is chosen, the architecture team decomposes the system into smaller, manageable units called components or modules.
- Each component is assigned specific responsibilities, and their interfaces and interactions are defined.
- This allocation process is crucial for achieving modularity, maintainability, and scalability.

Example: In an online learning platform:

- "Course Management" component handles course creation and updates.
- "User Profile" component manages user data and authentication.
- "Assessment" component handles quizzes and grading.

Validation

Using Scenarios

- Architects create detailed scenarios that represent important system behaviors and evaluate how well the proposed architecture supports them.
- **Purpose:** Test if architecture meets key use cases.
- **Approach:**
  - **Example Scenario:** "System must handle 10,000 concurrent users."
  - Evaluate if design supports this (e.g., load balancers, caching).
- **Output:** Scenario-based validation report.

Validation

Prototyping

- Prototyping involves building simplified versions of high-risk or complex components to validate architectural assumptions.
- **Purpose:** Validate high-risk components early.
- **Methods:**
  - **Throwaway Prototype** – Quick proof-of-concept.
  - **Evolutionary Prototype** – Iteratively refined into final product.
- **Output:** Prototype demonstrating critical functionality.

SA Design quality attributes

Availability

- Availability ensures the system remains operational and accessible when needed.
- This quality attribute focuses on minimizing downtime and providing graceful degradation during failures.

Implementation Considerations:

- Redundancy strategies (failover clusters, backup systems)
- Fault detection and recovery mechanisms
- Load balancing approaches
- Disaster recovery planning

SA Design quality attributes

Interoperability

- Interoperability enables the system to effectively communicate and exchange data with other systems.
- This is crucial in modern environments where systems rarely operate in isolation.

Implementation Considerations:

- Standardized communication protocols
- Data format compatibility
- API design principles
- Versioning strategies for interfaces

SA Design quality attributes

Performance

- Performance addresses how efficiently the system responds to user requests and handles workload.
- This includes both raw speed and resource utilization characteristics.

Implementation Considerations:

- Caching strategies
- Database optimization techniques
- Concurrency models
- Resource allocation policies

SA Design quality attributes

Security

- Security ensures protection against unauthorized access and malicious attacks while maintaining data confidentiality and integrity.

Implementation Considerations:

- Authentication and authorization mechanisms
- Data encryption standards
- Input validation strategies
- Audit logging requirements

SA Design quality attributes

Testability

- Testability refers to how easily the system can be verified to work as intended through various testing approaches.

Implementation Considerations:

- Modular Design for unit testing
- Logging and monitoring

SA Design Principles/Guidelines

- **Abstraction**
  - Hide complex details behind simplified interfaces.
  - Achieved through abstract classes/interfaces.
  - Focus on High-level design (what an object does).

E.g. The customer sees a "Place Order" button but doesn’t know how payment processing or inventory checks work.

SA Design Principles/Guidelines

- **Encapsulation**
  - Bundling of data (attributes) and methods (functions) that operate on the data into a single unit (a class), achieved via classes + access modifiers (getters/setters).
  - Protect internal data by only allowing external programs to access data through designed interface.
  - If we change the internal data structure or storage mechanism, then the external program will be affected

E.g. The "Order Module" internally handles:

- Validating the order, Deducting inventory, Charging payment.*(But exposes only a simple "Place Order" action to outsiders.)*

SA Design Principles/Guidelines

- **Information Hiding**
  - Try to hide the external interface or access methods from the internal design decisions.
    - Restrict access to internal (implementation) details (more robust than encapsulation).
  - Achieved via private/protected members.
  - Focus on Security and modularity.

E.g. The "Payment Module" hides:

- How transactions are logged.

SA Design Principles/Guidelines

- **Modularization**
  - Divide system into independent, manageable modules.
  - Example: An E-commerce application may be broken into modules for User Module, Inventory Module, Order Module, Shipping Module
- **Separation of Concerns**
  - Different functionalities should be handled by different modules

E.g.

- Order Module: Creates the order.
- Email Module: Sends the email.

➢ The "Order Module" doesn’t contain email-sending logic.

SA Design Principles/Guidelines

- **Cohesion and Decoupling**
  - Cohesion: Modules should have a single, well-defined purpose.
  - Decoupling: Modules should minimize dependencies.
- **Open Standard and Open Architecture**
  - Promotes interoperability and extensibility.
  - Open Standards: Use widely adopted protocols (e.g., HTTP, REST) and formats (JSON, XML).
  - Open Architecture: Design for extensibility (e.g., plugin systems, APIs).
- **Layering**
  - Organizes system into hierarchical layers (e.g., presentation, business logic, data).

Comparison: No-Hiding vs. Encapsulation vs. Information Hiding

Concept	Primary Goal	Access Control	Implementation Visibility	Impact on Clients	When to Use
No-Hiding	Full transparency of internal workings	No restrictions (Public everywhere)	All details exposed	High - Clients depend on internal structure	Only for simple data containers with no risk of misuse
Encapsulation	Group data and related methods together	Optional (can mix Public/Private)	Can be partially hidden	Medium - Clients interact via methods/properties	Common; used to organize related logic while allowing some flexibility
Information Hiding	Protect internal details from outside access	Strict (Private fields, limited Public methods)	Hidden behind an interface	Low - Clients rely only on public methods, not internals	Best practice for maintainable and secure OOP design

Architecture modeling language in development

- Requirement Description Language: specify software requirements.
- Component Description Language: Describe software components and their interfaces.
- Architectural Modeling Language: Model and describe the overall software architecture
- Framework Description Language: Describe the structure and behavior of software frameworks.
- Software Programming Language: Implement the actual code of the software system.
- System Integration Language: Describe how different systems are integrated.

Thanks !